

# IAPT: Benchmarking Small-Object Detection on Brain MRI: A Cerebral Microbleed Case Study

## Shared Core (All Students)

The following describes the shared foundational tasks, so to ensure a common experimental baseline and make the individual results directly comparable.

- 1. Dataset Preparation.** Download the VALDO Challenge 2021 Task 2 training dataset (72 subjects, T1/T2/T2\*-weighted MRI with binary microbleed segmentation masks). Perform standard preprocessing as required by nnU-Net: convert the data to the framework's expected directory structure and naming conventions, verify label integrity, and document any observations about the data (e.g., microbleed size distribution, number of microbleeds per subject, scan resolution variability).
- 2. Baseline Model Training.** Configure and train nnU-Net using its default 3D full-resolution configuration with five-fold cross-validation on the VALDO training set. This serves as the shared baseline against which each student's individual experiments are compared. All students must use the same data splits (nnU-Net generates these deterministically from the dataset) to ensure comparability.
- 3. Standard Evaluation Metrics.** Every student computes and reports the following metrics on the cross-validation results: (i) lesion-level sensitivity (proportion of ground-truth microbleeds detected), (ii) false positives per subject, (iii) lesion-level F1 score, and (iv) voxel-level Dice coefficient. A detection is counted as a true positive if a predicted connected component overlaps with a ground-truth microbleed annotation. These shared metrics provide the common language across all five reports.
- 4. Standardised Output Format.** Each student exports the baseline predictions as NIfTI files and evaluation metrics as structured CSV files following a common schema. This ensures that artefacts developed by individual students (e.g., the dashboard, the report generator, the benchmarking platform) can ingest outputs from the shared baseline and, where applicable, from each other's work.
- 5. Report and Deliverables.** Each student writes a report covering: background and motivation; dataset description; shared baseline methodology and results; their individual research thread (methodology, experiments, results, discussion); and a reflective conclusion. The 10-minute presentation should cover both the shared context and the individual findings. The video production should demonstrate the working artefact, including the baseline pipeline and the student's specific contribution.

## What Is This Project About?

**Cerebral microbleeds** are tiny deposits of blood breakdown products in the brain. They are clinically significant because their number and location are linked to stroke risk, cognitive decline, and small vessel disease. Radiologists detect them by visually scanning through hundreds of MRI slices, which is slow and error-prone. Two radiologists looking at the same scan will often disagree on the count. The goal of this project is to automate that detection using deep learning.

The same **MRI** scanner can produce different types of images by adjusting its pulse settings. T1-weighted images show anatomical structure (grey matter vs. white matter). T2-weighted images highlight fluid and pathology. T2\*-weighted images ("T2 star") are the critical sequence because they are sensitive to iron in old blood, microbleeds appear as small, round, dark spots on T2\*. The other sequences help distinguish real microbleeds from look-alikes such as blood vessel cross-sections.

✨ Open 3D Slicer and pre-load a VALDO subject. Toggle between T1, T2, and T2\* to show how microbleeds are visible on T2\* but invisible on the other sequences. Identify a microbleed and put up side-by-side with a vessel cross-section to visualise the discrimination.

## The Dataset

The VALDO Challenge was a competition held at the MICCAI 2021 medical imaging conference. Task 2 of the challenge focused specifically on cerebral microbleed detection. The organisers released a publicly available training dataset of 72 subjects, each with T1, T2, and T2\* brain MRI volumes plus a binary segmentation mask in which every microbleed has been manually annotated by expert neuroradiologists. The data is in NIfTI format (.nii.gz), which is the standard file format in neuroimaging.

In practice for each subject there are four files: one for each MRI sequence and one for the segmentation mask. The mask is a volume of the same dimensions as the images where every voxel is labelled either 0 (background) or 1 (microbleed).

✨ Use the 3D Slicer and load a subject's T2\* volume alongside its segmentation mask overlaid in colour. Zoom into a labelled microbleed to check what the ground truth looks like at voxel level.

The licence: the dataset is released under CC BY-NC-SA 4.0, which permits noncommercial research use. An MSc or BSc dissertation at the University of Malta qualifies. No ethics application is needed because the data is already anonymised and publicly released.

## 1. Dataset Preparation for nnU-Net

**nnU-Net's expectations:** nnU-Net version 2 is designed to be self-configuring, but it requires the input data to follow a strict directory layout. If the naming or structure is wrong, the framework will either fail outright or produce incorrect results silently. There are three things that you must get right: the folder structure, the file naming convention, and the dataset.json metadata file.

This is the required folder structure:

```

nnUNet_raw/
  Dataset001_VALDO/
    imagesTr/
      VALDO_001_0000.nii.gz      ← T1 for subject 001
      VALDO_001_0001.nii.gz      ← T2 for subject 001
      VALDO_001_0002.nii.gz      ← T2* for subject 001
      VALDO_002_0000.nii.gz      ← T1 for subject 002
      ...
    labelsTr/
      VALDO_001.nii.gz           ← segmentation mask for subject 001
    VALDO_002.nii.gz             ← segmentation mask for subject 002
    ... dataset.json              ← metadata
  descriptor

```

**The naming rules are very important:** each image file has a four-digit modality suffix (\_0000, \_0001, \_0002). The suffix is how nnU-Net knows which modality is which. The label file uses the same case identifier (e.g., VALDO\_001) but without any modality suffix. Every image file for the same subject must share exactly the same case identifier. If there is a single typo in a filename, nnU-Net will either ignore that case or crash.

**⚠ Common pitfall:** It is strongly recommended that you write a short Python script that reads the original VALDO directory, renames everything systematically, and places it into the nnU-Net structure. This script also serves as documentation of exactly how the conversion was done.

The **dataset.json** file should be created so that nnU-Net “knows” about the dataset. Here is a concrete example:

```

{
  "channel_names": {
    "0": "T1",
    "1": "T2",
    "2": "T2star"
  },
  "labels": {
    "background": 0,
    "microbleed": 1
  },
  "numTraining": 72,
  "file_ending": ".nii.gz"
}

```

The **channel\_names** maps the modality suffixes to human-readable names. **labels** defines the segmentation classes. **numTraining** must match exactly the number of cases in the imagesTr folder. **file\_ending** tells nnU-Net what file extension to expect.

**Important:** Write a script, do not do it by hand, and verify everything before moving on.

## 2. Walk Through Label Verification

Before training a model, you must verify that the labels are correct and consistent. This is not optional, since training on corrupted or misaligned labels will produce a model that appears to work but produces meaningless results.

You should write a verification script that checks three things:

1. **Label value integrity:** every voxel in every label file should be either 0 or 1. If any other values appear (e.g., 2, 255, or floating-point artefacts), the label is corrupt.
2. **Spatial alignment:** the label volume must have exactly the same dimensions (shape) and spatial mapping (affine matrix) as the corresponding image volumes. If these do not match, the labels and images are misaligned and the model will learn from the wrong locations.
3. **Connected-component statistics:** run `scipy.ndimage.label` on each mask to count the number of discrete microbleed objects per subject. This produces the basic dataset statistics: how many microbleeds exist per subject (the distribution across the 72 cases), and what their individual sizes are in voxels.

**You should document:** a table or histogram showing the microbleed count distribution across subjects (e.g., how many subjects have 0, 1–5, 5–20, or 20+ microbleeds), and a histogram of individual microbleed sizes in mm<sup>3</sup> (converting from voxel count using the voxel spacing from the NIfTI header). These statistics belong in the shared baseline section of their final report.

**Milestone check:** At the end of this step, you should have: (a) the VALDO data reorganised into nnU-Net format, (b) a `dataset.json` file, (c) a verification script that confirms all labels are valid and aligned, and (d) a summary of dataset statistics.

## 3. nnU-Net and the Training Process

**What is nnU-Net and why it was chosen:** nnU-Net (no-new-Net) is a framework developed at the German Cancer Research Center (DKFZ) that automates nearly all of the design decisions in medical image segmentation. It analyses the dataset's properties (image sizes, voxel spacing, intensity distributions, class frequencies) and automatically determines the preprocessing pipeline, network architecture, patch size, batch size, and training schedule. You do not need to design the network, nnU-Net does. You are responsible to set up the data correctly, run the pipeline, and understand what it is doing and why.

These are the three commands that you will need to run:

**Command 1: Plan and preprocess.** This command analyses the dataset and generates a preprocessing plan. It computes target spacing, intensity normalisation statistics, patch sizes, and architecture configurations. It then preprocesses all 72 training cases accordingly. This step is CPU-bound, not GPU-bound, and should roughly take between 30–60 minutes.

```
nnUNetv2_plan_and_preprocess -d 001 --verify_dataset_integrity
```

✨ When you execute this command you can view the output log to see the kind of information nnU-Net reports: target spacing, patch size, architecture details. This is what is meant by “self-configuring”.

**Command 2: Train.** This command trains a single fold of the five-fold cross-validation. You need to run it five times, once for each fold (0 through 4). The five-fold cross-validation means the 72 subjects are split into five groups; for each fold, four groups are used for training and one for validation. By the end of all five folds, every subject has been used exactly once as a validation case.

```
nnUNetv2_train 001 3d_fullres 0 # Fold 0
nnUNetv2_train 001 3d_fullres 1 # Fold 1
nnUNetv2_train 001 3d_fullres 2 # Fold 2
nnUNetv2_train 001 3d_fullres 3 # Fold 3
nnUNetv2_train 001 3d_fullres 4 # Fold 4
```

**Compute requirements:** each fold trains for 1,000 epochs (where each epoch is 250 iterations, not a full dataset pass). On a T4 GPU (Google Colab Pro) this takes roughly 10–20 hours per fold; on an RTX 3090 or 4090 it may take 6–12 hours. Five folds therefore might require 40–100 total GPU-hours. On Colab, session timeouts are a real issue. It is therefore recommended to use Colab Pro+ for longer sessions. nnU-Net checkpoints are saved automatically, so a timed-out session can be resumed.

**⚠ Important:** You need to use the same data splits. Since nnU-Net generates splits deterministically from the dataset, this happens automatically as long as everyone uses exactly the same dataset folder with the same case identifiers. Do not shuffle, rename, or reorder cases.

**Command 3: Predict.** Once all five folds are trained, this command runs inference on the validation cases. Because each subject appears in exactly one fold's validation set, the combined output covers all 72 subjects with predictions the model has never seen during training.

```
nnUNetv2_predict -i INPUT_FOLDER -o OUTPUT_FOLDER
\ -d 001 -c 3d_fullres -f 0 1 2 3 4 --
save_probabilities
```

**What is the --save\_probabilities flag:** this saves the raw probability maps (not just the thresholded binary predictions). Several of the individual development threads (the post-processing toolkit, the dashboard) will need these probability maps. Without this flag, you would need to re-run inference later.

## 4. The Evaluation Metrics

Once predictions exist, you must compute four metrics. Here is an explanation of what each measures and why it matters.

**Lesion-level sensitivity (recall):** of all the microbleeds in the ground truth, what fraction did the model detect? A detection counts as a true positive if any predicted connected component overlaps with a ground-truth microbleed by at least one voxel. This measures how many real microbleeds the model finds. A sensitivity of 0.80 means the model catches 80% of microbleeds but misses 20%.

**False positives per subject:** how many predicted connected components have no overlap with any ground-truth microbleed? This is reported as an average per subject. A model with high sensitivity but also high false positives would flood a radiologist with false alarms, which is clinically useless.

**Lesion-level F1 score:** the harmonic mean of lesion-level precision and sensitivity. This is the single number that balances finding all microbleeds against not flagging too many false positives. It is the primary metric for comparing configurations.

**Voxel-level Dice coefficient:** the overlap between predicted and ground-truth segmentation masks at the voxel level. This measures how well the model delineates the shape and extent of each microbleed, not just whether it found it. For very small objects, Dice can be misleadingly low even when detection is good, because a single misplaced voxel has an outsized effect.

You will need to write a short evaluation script (or adapt an existing one) that performs connected-component analysis on both the prediction and ground truth, matches components by overlap, and computes these four metrics. The outputs should be saved as a structured CSV file with one row per subject and columns for each metric.

## 5. The Standardised Output Format

This step is short but critical. The individual development threads depend on being able to load each other's outputs. If everyone saves files in different formats or with different naming conventions, no one's tool will work with anyone else's data.

This is the shared output standard:

1. **Predictions:** NIfTI files (.nii.gz), one per subject, with the same case identifier as the original dataset (e.g., VALDO\_001.nii.gz). Binary segmentation masks (0/1) at the default nnU-Net threshold.
2. **Probability maps:** NumPy arrays (.npz), one per subject, containing the raw softmax probabilities for the microbleed class.
3. **Metrics:** a single CSV file (evaluation\_results.csv) with columns: subject\_id, sensitivity, false\_positives, f1\_score, dice, num\_gt\_microbleeds, num\_pred\_microbleeds. One row per subject.